



Feature Store, Experiments & Model Registry

Session 2 — Deep Dive into Production ML on Snowflake

MAY 2026 · SESSION 2



Chris Caudill

Senior AI/ML Architect

Snowflake

PRESENT

Senior AI/ML Architect

Snowflake

End-to-end ML platform strategy, customer architecture, production ML enablement.

5 YEARS

Senior ML Engineer

Amazon Web Services (AWS)

Enterprise-scale MLOps, Fuels Price Optimization, Agentic Systems.

10 YEARS

Senior Data Scientist

ExxonMobil

Investigative NLP, Market Signal Detection, Summarization Modeling.



SERIES

Session Series: Machine Learning on Snowflake

A 4-part deep dive into production ML capabilities on Snowflake.



SESSION 1 · COMPLETE

End-to-End ML Overview

Platform overview, production-grade ML, live demo.

SESSION 2 · NOW

Feature Store, Experiments & Registry

Entities, FeatureViews, point-in-time correctness, experiment tracking, model registry.

SESSION 3

Training, HPO & Model Registry

Distributed training, HPO, registry RBAC.

SESSION 4

Deployment, Monitoring & MLOps

Model Serving, Observability, CI/CD.

 AGENDA

Today's Agenda

Three pillars of production ML management — all native to Snowflake.



Part 1 — Feature Store

- › Feature engineering with Snowpark
- › Online & offline stores
- › Point-in-time correctness
- › Automated refresh via Dynamic Tables
- › Dataset versioning



Part 2 — Experiment Tracking

- › Load versioned datasets
- › Establish a baseline run
- › Vary hyperparameters
- › Compare metrics side-by-side
- › Select the best run

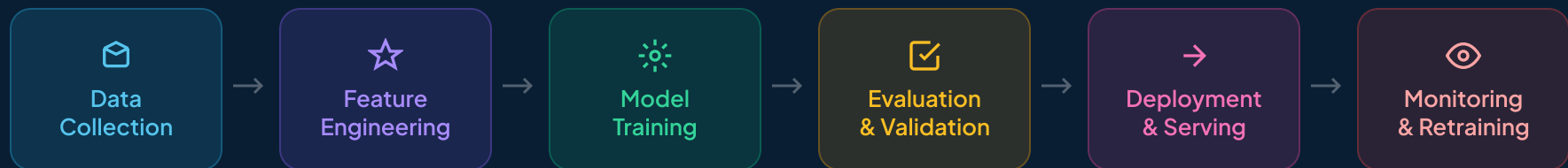


Part 3 — Model Registry

- › Register & version models
- › End-to-end lineage
- › Aliases & default version
- › Tagging & metadata
- › Promotion workflow

 LIFECYCLE

The Standard Machine Learning Lifecycle



Each stage introduces complexity, tooling fragmentation, and data movement risk.

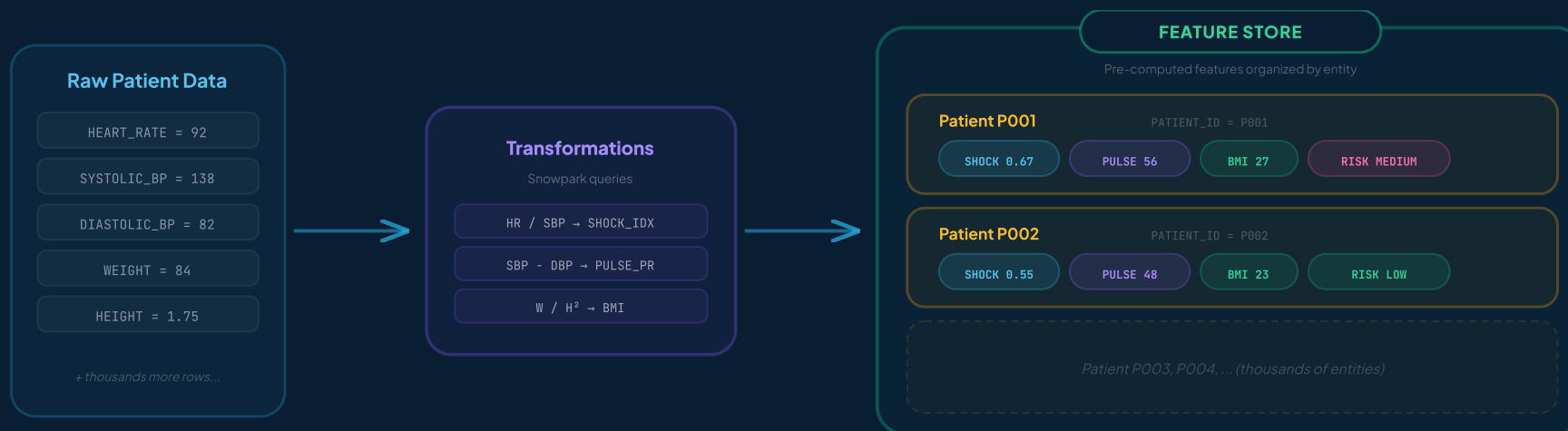
Part 1

Feature Engineering & Feature Store

Encode domain expertise once. Serve everywhere. Never skew.

What Is a Feature Store?

Raw data goes in. **Pre-computed, ready-to-use features** come out — organized by patient, refreshed automatically, and served to any model that needs them.



Point-in-Time Correctness

No data leakage in training



Shared Across Teams

Any model can reuse features



Auto-Refresh

Dynamic Tables on a schedule



Immutable Datasets

Versioned, reproducible snapshots



Zero Skew

Same features for train & serve



Automatic Lineage

Source → features → model

Feature Store Core Concepts

A managed layer between raw data and ML models — owns transformation logic, keeps features fresh, and serves the **same features** to both training and inference. In Snowflake, a Feature Store is simply a **schema**.

📌 Entity

The **real-world object** your features describe. Defined by one or more **join keys** (e.g. `PATIENT_ID`).

Key idea: A single Entity can link *many* Feature Views together. It's the glue that lets you combine features from different sources into one training dataset.

👁️ Feature View

The **transformation + materialization**. A Snowpark query wrapped into a **Dynamic Table** that auto-refreshes on a schedule.

Key idea: A Feature View references one or more Entities via their join keys. It defines *what* to compute and *how often* to refresh.



Features Can Be Complex

Features aren't limited to simple ratios. The most powerful features encode **domain expertise** and even the output of **other ML models**.



Cancer Risk Score

A feature that wraps a **Cox proportional-hazards model** combining age, family history, genomic markers, and lab trends into a single **10-year cancer risk probability**. Too expensive to compute at inference time — so it's pre-materialized.

```
-- Feature View query (simplified)
SELECT PATIENT_ID, TIMESTAMP,
       cancer_risk_udf(
         age, family_hx, brca_status,
         smoking_years, bmi_trend_12mo,
         crp_level, wbc_trend_6mo
       ) AS CANCER_RISK_10YR,
       CASE WHEN CANCER_RISK_10YR > 0.20
            THEN 'HIGH'
            WHEN CANCER_RISK_10YR > 0.10
            THEN 'MODERATE'
            ELSE 'LOW'
       END AS CANCER_RISK_TIER
FROM PATIENT_LONGITUDINAL;
```

Key insight: the cancer risk score is itself an ML model output — but when consumed by a downstream readmission predictor, it's *just another feature column*.

MORE COMPLEX FEATURE EXAMPLES



90-Day Rolling Readmission Rate

Window function scanning months of encounter history per patient — joins 2 tables, $O(n)$ per patient.



Medication Interaction Score

Cross-join of active prescriptions against a contraindication graph — combinatorial complexity.



Charlson Comorbidity Index

Weighted sum across 17 disease categories from full diagnosis history — standardized clinical risk metric.



NLP-Derived Social Determinants

Cortex LLM extraction from clinical notes — housing stability, transportation access, food security.

COMPUTE ON-THE-FLY

5–30s

UDF + joins + window scans

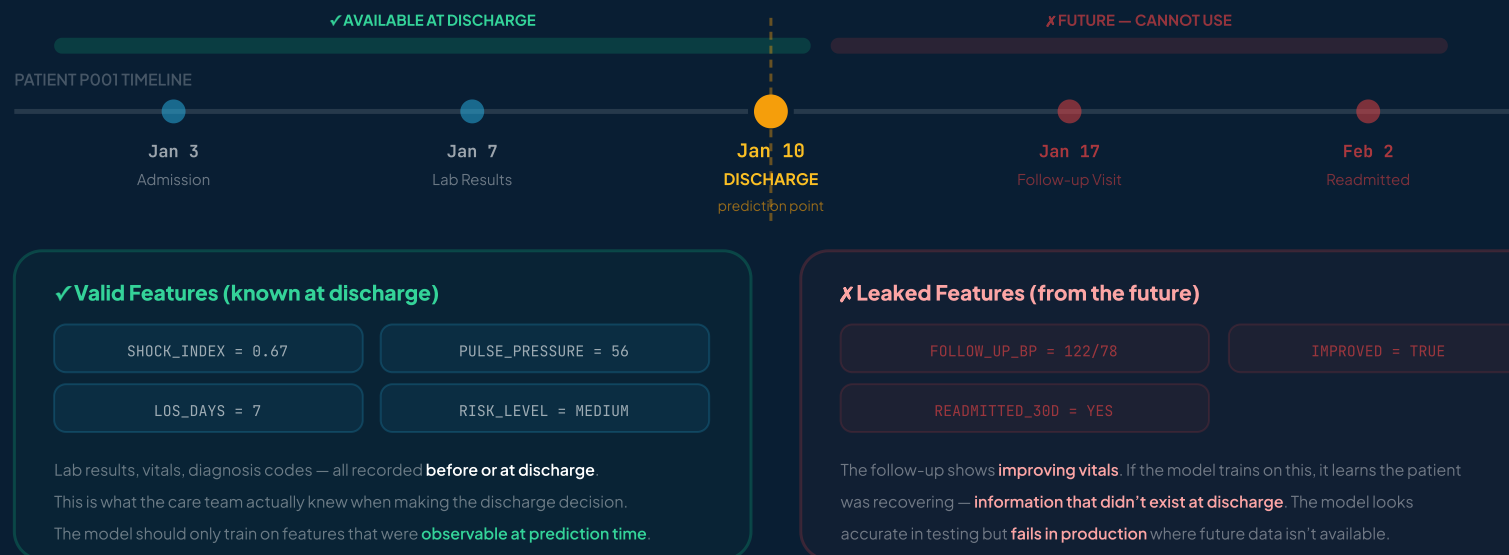
PRE-MATERIALIZED LOOKUP

~30ms

Online Store key lookup

Why Point-in-Time Correctness Matters

We want to predict **30-day readmission risk at discharge**. The model must only see what the care team knew *at that moment* — nothing from after the patient left.



The rule: When training a model to predict at discharge, features must come from **≤ discharge time**. Snowflake's `spine_timestamp_col` enforces this automatically via an AS-OF join — no manual filtering needed.

🕒 EXAMPLE

Point-in-Time by Example

Walk through exactly which feature rows get joined for each spine entry.

SPINE (PREDICTION EVENTS)

PATIENT_ID	TIMESTAMP
P001	Jan 15
P002	Feb 20

AS-OF
JOIN
➔

FEATURE VIEW (DYNAMIC TABLE)

PATIENT_ID	TIMESTAMP	SHOCK_IDX	RISK	USED?
P001	Jan 10	0.61	MED	✓
P001	Jan 20	0.74	HIGH	✗
P002	Feb 18	0.55	LOW	✓
P002	Feb 10	0.68	MED	older

The “**who + when**” of each prediction event.

Only features with **ts** ≤ **spine ts** are eligible. The **most recent** eligible row wins.

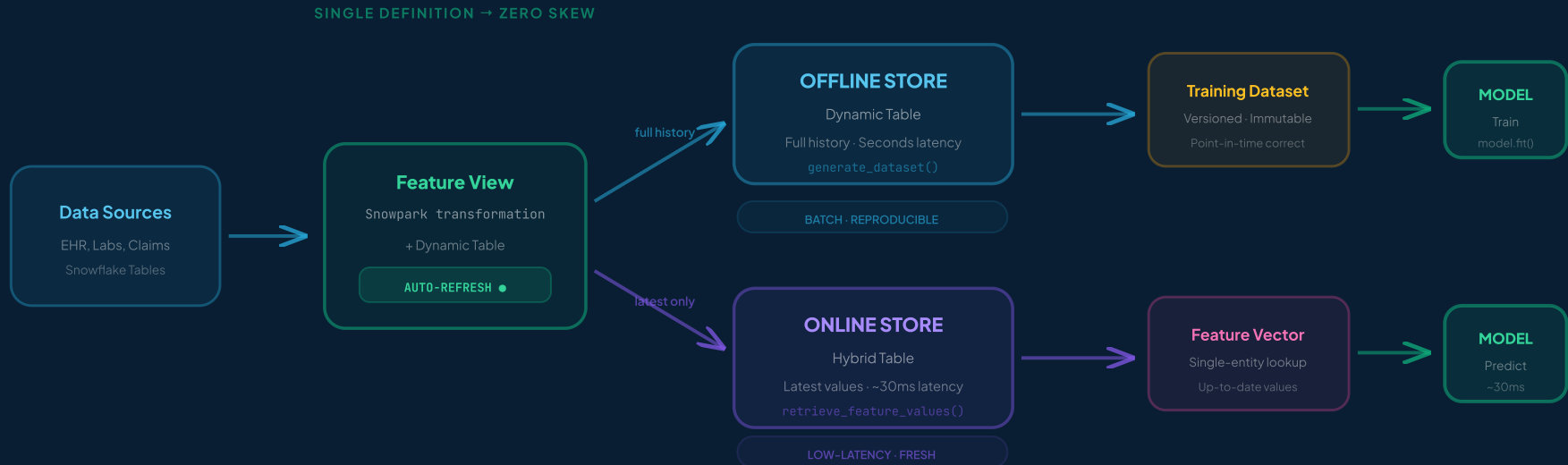
RESULT: TRAINING DATASET (POINT-IN-TIME CORRECT)

PATIENT_ID	SPINE_TS	FEATURE_TS	SHOCK_INDEX	RISK_LEVEL	WHY THIS ROW?
P001	Jan 15	Jan 10	0.61	MED	Jan 10 ≤ Jan 15 ✓ Jan 20 > Jan 15 excluded
P002	Feb 20	Feb 18	0.55	LOW	Feb 18 ≤ Feb 20 ✓ Feb 10 also valid but older

Rule: For each spine row, join the **most recent feature row** where **feature_ts** ≤ **spine_ts** and **patient_id** matches. Future features are **never** included.

Offline vs. Online Feature Serving

One Feature View definition, two serving paths. The **offline store** retains full history for training; the **online store** keeps only the latest values for low-latency inference.



Offline Store = Dynamic Table

Stores **complete feature history**. Used for building *training datasets* with point-in-time correctness. Latency is seconds — acceptable for batch workloads.

Online Store = Hybrid Table

Stores only the **latest feature values** per entity. Optimized for *single-key lookups* at ~30ms — fast enough for real-time inference behind an API.



CODE

Feature Store in Action

KEY CONCEPTS



FeatureStore

= a Snowflake schema. No external infra.



Entity

Join key linking features across views.



FeatureView

Transformation + Dynamic Table auto-refresh.



Dataset

Immutable, versioned, point-in-time snapshot.

ZERO TRAINING-SERVING SKEW

Same Feature View serves both **generate_dataset()** at training and **retrieve_feature_values()** at inference.

FEATURE_STORE_SETUP.PY

```
# Initialize Feature Store (just a schema)
from snowflake.ml.feature_store import (
    FeatureStore, FeatureView, Entity, CreationMode
)

fs = FeatureStore(session, database="ML_DB",
    name="HEALTHCARE",
    default_warehouse="ML_WH",
    creation_mode=CreationMode.CREATE_IF_NOT_EXIST)

# Register entity (join key for all features)
patient = Entity("PATIENT", join_keys=["PATIENT_ID"])
fs.register_entity(patient)

# Engineer features with Snowpark (server-side)
feature_df = raw_df.with_column("SHOCK_INDEX",
    F.col("HEART_RATE") / F.col("SYSTOLIC_BP")
).with_column("PULSE_PRESSURE",
    F.col("SYSTOLIC_BP") - F.col("DIASTOLIC_BP"))

# Register Feature View (creates Dynamic Table)
fv = FeatureView("PATIENT_FEATURES", [patient],
    feature_df, refresh_freq="1 minute",
    timestamp_col="TIMESTAMP")
fs.register_feature_view(fv, version="v1", block=True)

# Generate point-in-time training dataset
dataset = fs.generate_dataset(
    spine_df=spine, features=[fv],
    spine_timestamp_col="TIMESTAMP",
    name="PATIENT_RISK_DS", version="v1")
```

Part 2

Experiment Tracking

Log runs, compare metrics, select winners — no external tracking server.



EXPERIMENTS

Experiment Tracking Overview

Native experiment tracking stored as Snowflake metadata — governed by the same RBAC as your data.





CODE

Experiment Tracking in Action

API METHODS

`set_experiment()` Create/connect
`start_run()` Context manager
`log_params()` Hyperparameters
`log_metrics()` Evaluation scores
`log_model()` Wraps Registry
`log_artifact()` Files/dirs

NO TRACKING SERVER

Experiments stored as Snowflake metadata. Same credentials, same RBAC, same Snowsight UI.

VIEW IN SNOWSIGHT

AI & ML → **Experiments** for a visual comparison of all runs.

EXPERIMENT_TRACKING.PY

```
# Initialize experiment tracking
from snowflake.ml.experiment import ExperimentTracking

exp = ExperimentTracking(session)
exp.set_experiment("PATIENT_RISK_EXPERIMENT",
                  database=db, schema=schema)

# Run 1: Baseline
with exp.start_run(run_name="baseline_rf") as run:
    run.log_params({
        "model": "RandomForest",
        "n_estimators": 100,
        "class_weight": "balanced",
    })
    model = pipeline.fit(X_train, y_train)
    preds = model.predict(X_test)
    run.log_metrics({
        "accuracy": accuracy_score(y_test, preds),
        "f1_weighted": f1_score(y_test, preds,
                                average="weighted"),
    })

# Compare all runs
runs_df = session.sql(
    f"SHOW RUNS IN EXPERIMENT {DB}.{SCHEMA}"
    f".PATIENT_RISK_EXPERIMENT"
).to_pandas()
```

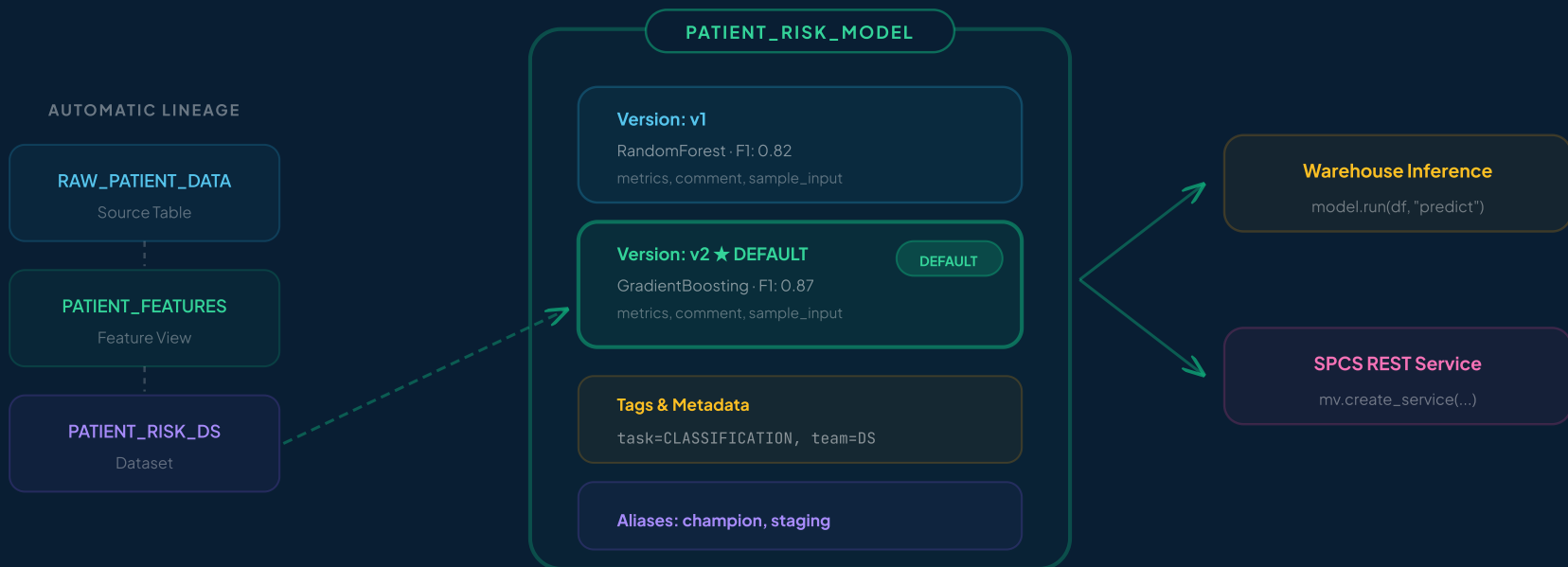

Part 3

Model Registry

Version, trace, tag, and promote models — all within Snowflake governance.

Model Registry **Architecture**

Models are first-class Snowflake objects with versioning, lineage, and RBAC built in.



Model Registry in Action

REGISTRY CAPABILITIES

- ✓ Named versions (semantic versioning)
- ✓ Default version abstraction
- ✓ Metrics attached to version
- ✓ Automatic upstream lineage
- ✓ Tags for team/domain/task
- ✓ Warehouse or SPCS inference
- ✓ Built-in explain() for SHAP

PROMOTION WORKFLOW

Check metrics against thresholds → set as default → deploy to service. Bad models never reach production.

MODEL_REGISTRY.PY

```
# Register model in the Snowflake Model Registry
from snowflake.ml.registry import Registry
from snowflake.ml.model.task import Task

registry = Registry(session, database_name=db,
                    schema_name=schema)

# One call: serialize, store, capture lineage
mv = registry.log_model(
    model=best_pipeline,
    model_name="PATIENT_RISK_MODEL",
    version_name="v2",
    sample_input_data=X_train.head(10),
    metrics={"f1": 0.87, "accuracy": 0.89},
    task=Task.TABULAR_MULTI_CLASSIFICATION,
    comment="GradientBoosting with tuned LR",
)

# Tag the model for discoverability
mv.set_tag("team", "clinical_ds")
mv.set_tag("domain", "patient_risk")

# Promotion: set as default if thresholds pass
model_ref = registry.get_model("PATIENT_RISK_MODEL")
if mv.get_metrics()["f1"] >= 0.75:
    model_ref.default = "v2"

# View lineage in Snowsight: AI & ML > Models
lineage = mv.lineage() # upstream provenance
```

End-to-End Lineage

Automatically captured — from source table to production predictions. No manual annotation.



Captured automatically via `sample_input_data` in `log_model()` — no manual annotation needed.

View in Snowsight → AI & ML → Models → Lineage tab



RECAP

Session 2 Summary

Everything we covered today — all native to Snowflake, no external tools required.



Feature Store

- › Feature Store = Snowflake schema
- › Dynamic Tables auto-refresh
- › Point-in-time correctness
- › Online (Hybrid) + Offline (DT)
- › Immutable dataset versions
- › Zero training-serving skew



Experiment Tracking

- › No external tracking server
- › Log params, metrics, artifacts
- › Compare runs via SQL
- › View in Snowsight UI
- › Same RBAC as your data
- › Select best run programmatically



Model Registry

- › Named versioning
- › Default version = production
- › Automatic end-to-end lineage
- › Tags & aliases for discoverability
- › Promotion gates with thresholds
- › Deploy to Warehouse or SPCS

Next session: **Distributed Training, HPO & Advanced Deployment**